

SimpleScalar v2.0 Tutorial

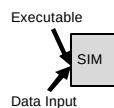
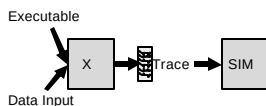
Univ of Wisc - CS 752
Dana Vantrease
(leveraged largely from Austin &
Burger)

Simulator Basics

- What is a simulator?
 - Tool that runs and emulates the behavior of a computing device
- Why use a simulator?
 - Flexible and cheap
- Why not use a simulator?
 - Slow
 - Correctness?

Types of Simulators

- Trace Driven
 - A trace of the instructions executed by a processor running the application is recorded in a file and then used by the simulator
- Execution Driven
 - The executable is run directly, and the instruction stream is determined by the execution path taken.



Execution vs Trace

- Trace
 - + Easy to Implement
 - Requires large disk files to store instruction stream
 - Doesn't include speculated/squashed instructions
- Execution
 - Hard to Implement
 - + Allows access to all data produced and consumed during program execution
 - +/- Execution requires inclusion of instruction set emulator and an I/O emulation module

Important SimpleScalar Websites

- <http://www.simplescalar.com>
- <http://www.cs.wisc.edu/mscalar/ss>

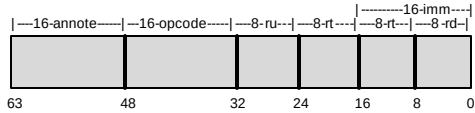


SimpleScalar Intro

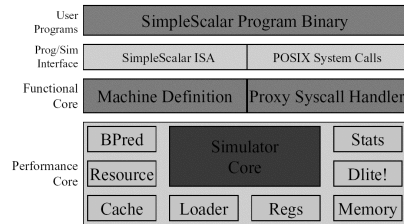
- Developed by Wisconsin Badgers
- Execution Driven
- Collection of microarchitectual simulators that emulate the microprocessor at different levels of details and configurations (in-order, out-of-word, etc)
- ISA is a derivative of MIPS ISA
 - Takes C or Fortran binaries compiled for SimpleScalar architecture
 - Compiler is based on GNU GCC compiler
 - `ssbig-na-strix-gcc`
 - `ssbig-na-sstrix-f77`

SimpleScalar Instruction Set

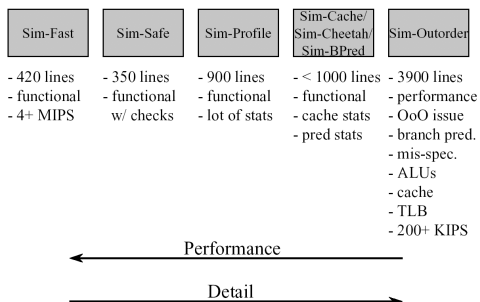
- MIPS-based + more addressing modes
- Bi-endian
- 64 bit instruction encoding
 - 16-bits extra
 - hints
 - new instructions
 - annotations



Simulator S/W Architecture



Simulation Suite Overview



Sim-Fast

- No timing
- Optimized for Speed
- Serial instruction execution
- Does not account for the behavior of any part of the microarchitecture (pipelines, caches, etc)



Sim-Safe

- Similar to Sim-Fast (but slightly slower)
- On all memory operations checks
 - Memory access permission
 - Memory alignment
- Can be good for debugging sim-fast crashes



Sim-Profile

- Profiles by symbol and by address
- Keeps track of and reports
 - Dynamic instruction counts
 - Instruction class counts
 - Usage of address modes
 - Profiles of the text & data segment



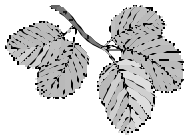
Sim-Cache/Sim-Cheetah

- Emulates multiple levels of instruction and data caches
 - Variable sizes
 - Variable organizations
- Do not take into account access times, so suitable only for studying miss-rates



Sim-Bpred

- Simulates different branch prediction schemes
- Reports:
 - Prediction hit
 - Miss Rate
- Does not simulate accurately the effect of branch prediction on execution time

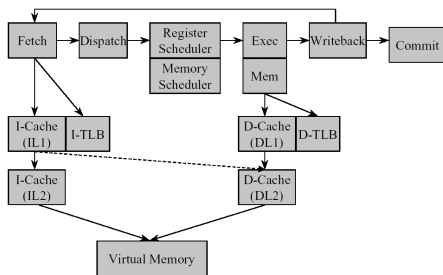


Sim-OutOrder

- Out-of-order instruction issue
- Keeps track of event timing
- Detailed
 - Branch prediction
 - Caches
 - External memory
 - Various Configurations



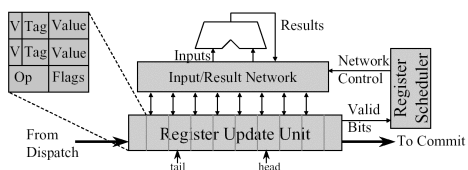
Sim-OutOrder HW arch



Sim-OutOrder Register Update Unit

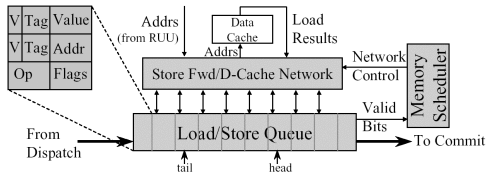
- Take advantage of 1-to-1 correspondence between Tomasulo's tag field and reservation unit.
→ combine into Reservation Station/Tag Unit (RSTU)
- RSTU can hold instruction results (i.e. reorder buffer)
→ FIFO/Circular Queue Register Update Unit (RUU)

Sim-OutOrder - RUU



- RUU handles register synchronization/communication
 - unifies reorder buffer and reservation stations
 - managed as a circular queue
 - entries allocated at Dispatch, deallocated at Commit
 - out-of-order issue, when register and memory deps satisfied
 - memory dependencies resolved by load/store queue (LSQ)

Sim-OutOrder – Load/Store Queue



- LSQ handles memory synchronization/communication
 - contains all loads and stores in program order
 - load/store primitives really, address calculation is separate op
 - effective address calculations reside in RUU (as ADD insts)
 - loads issue out-of-order, when memory deps known satisfied
 - load addr known, source data identified, no unknown store address

Miss Status Holding Registers

- Exploits spatial locality of sequential misses with net
- MSHR miss: allocate an MSHR, initialize one target
- MSHR hit: allocate one target
- When response returns, fire all targets
- If no available MSHRs or targets (L1 only)
 - Place load back in issue ready queue
 - Prevent store from committing
 - Continue stalling I-fetch



Sim-OutOrder – Main Loop

```
for (;;) {
    ruu_commit();
    ruu_writeback();
    lsq_refresh();
    ruu_issue();
    ruu_dispatch();
    ruu_fetch();
}
```

- main simulator loop is implemented in `sim_main()`
- walks pipeline from Commit to Fetch
 - backward pipeline traversal eliminates relaxation problems, e.g., provides correct inter-stage latch synchronization
- loop is exited via a `longjmp()` to `main()` when simulated program executes an `exit()` system call

Sim-OutOrder Stage Implementation

- `Fetch(ruu_fetch())`
 - Fetch instruction from cache/memory
 - Queue in Instruction Fetch Queue
 - Probe branch predictor for cache line to access not cycle from one lcache line
- `Dispatch(ruu_dispatch())` – decode, register renaming
 - Fetch from Instruction Fetch Queue
 - Decode Instructions
 - Enter Instructions into RUU and LSQ



Sim-OutOrder Stage Implementation (cont)

- `Scheduler(ruu_issue() & lsq_refresh())` - instruction wakeup, selection, issue
 - Insert instructions with registers ready to ready queue
 - Loads with all memory inputs ready (forwarding or D-Cache)
- `Execute(ruu_execute())` – goto Functional Units
 - Get instructions that are ready
 - Reserve free functional unit
 - Schedule writeback event using operational latency of functional unit

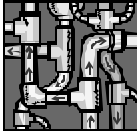


Sim-OutOrder Stage Implementation (cont)

- `WriteBack(ruu_writeback())` wakes up finished instructions, detects mispredicts
 - Get finished instructions
 - If mispredicted branch, recover RUU & architected state
 - Wakeup instructions on finished instruction's output dependence chains
- `Commit(ruu_commit())` – in-order retirement, D-cache store commits, D-TLB miss handling
 - While of of RUU/LSQ ready to commit
 - Service D-TLB misses
 - Retire store to D-cache
 - Update register file and rename table
 - Reclaim RUU/LSQ resources of committed instructions

Sim-OutOrder Pipetraces

- Detailed history of all instructions executed
 - Instruction fetch
 - Retirement
 - Pipeline stage transitions
- Displays pipeline for each cycle of execution traced
- Sim-OutOrder Command line option:
 - `-ptrace <ptrace_file>`
 - `<end_range> | start_range:end_range>`
- View ptrace_file with:
 - `pipeview.pl <ptrace_file>`



{sim-cache, sim-profile, sim-outorder} PC-Based Stat Profiles

- ❑ Produces text segment profile for any integer statistical counter
- ❑ Command line
 - `pcstat <sim_num_insn | sim_num_regs, il1.misses | bpred_bimod.misses | ... >`
- ❑ View with
 - `textprof.pl <dis_file> <sim_output> <sim_num_insn | sim_num_regs, il1.misses | bpred_bimod.misses | ... >`
 - `objdump -dl <application name> >| <dis_file>`

Misc Useful Stuff

- `misc.[hc]`
 - `fatal`, `panic`, `warn`, `info`, `debug`, `elapsed time`, `getcore`
- `stats.[hc]`
 - Provides counters, expressions, distributions database

Machine Definition File (ss.def)

- a single file describes all aspects of the architecture
 - used to generate decoders, dependency analyzers, functional components, disassemblers, appendices, etc.
 - e.g., machine definition + ~30 line main = functional sim
 - generates fast and reliable codes with minimum effort
- instruction definition example:

```

DEFINST (ADDI,
disassembly → "addi",
template → IntALU,
FU req's → DGPR (RT), NA,
output deps → SET_GPR (RT, GPR (RS) + IMM)
                                semantics
                                0x41,
                                "t,s,i",
                                inst flags → F_ICOMP | F_IMM,
                                DGPR (RS), NA, NA,
                                input deps

```

Simulator Core Interface

```

/* register simulator-specific options */
void sim_reg_options(struct opt_odb_t *odb);

/* check simulator-specific option values */
void sim_check_options(struct opt_odb_t *odb, int argc, char **argv);

/* register simulator-specific statistics */
void sim_reg_stats(struct stat_odb_t *adb);

/* initialize the simulator */
void sim_init(void);

/* print simulator-specific configuration information */
void sim_aux_config(FILE *stream);

/* start simulation, program loaded, processor precise state initialized */
void sim_main(void);

/* dump simulator-specific auxiliary simulator statistics */
void sim_aux_stats(FILE *stream);

/* un-initialize simulator-specific state */
void sim_uninit(void);

```

- called in this order (from main.c)
- defined in sim.h

Running SimpleScalar

- <http://www.cs.wisc.edu/mscalar/ss/>

File	Contents
TR_1342.ps	The technical report documenting release 2.0 of the tool suite
simplesim.tar	The simulator code (required)
simpleutils.tar	The binary utilities (recommended)
simpletools.tar	The compiler, assembler, libraries (optional)
simplebench.big.tar	Precompiled, big-endian SS SPEC95 binaries (optional)
simplebench.little.tar	Precompiled, little-endian SS SPEC95 binaries (optional)
INSTALL	Installation instructions for the general release
COPYRIGHT	Duplication, distribution, and use restriction

Running Simulators

- Decompress tar file:
tar -xvf simplescalar.tar
tar -xvf simplebench.big.tar
etc...
- Directions for making simulators in
simplescalar-2.0/README
 - Compiles on NOVAS out of the box
 - Does not compile on TUXs out of the box
- Run executable (sim-outorder, sim-safe,
etc) without any arguments to find command
format

Sample Sim-OutOrder Output

```
sim: ** simulation statistics **
sim_num_insn      110988 # total number of instructions committed
sim_num_refs      45449 # total number of loads and stores committed
sim_num_loads      26216 # total number of loads committed
sim_num_stores     19233.0000 # total number of stores committed
sim_num_branches   23598 # total number of branches committed
sim_elapsed_time   1 # total simulation time in seconds
sim_inst_rate     110988.0000 # simulation speed (in insts/sec)
sim_total_insn     120599 # total number of instructions executed
sim_total_refs     48421 # total number of loads and stores executed
sim_total_loads     28165 # total number of loads executed
sim_total_stores   20256.0000 # total number of stores executed
sim_total_branches 25357 # total number of branches executed
sim_cycle          90812 # total simulation time in cycles
sim_IPC            1.2222 # instructions per cycle
sim_CPI            0.8182 # cycles per instruction
sim_exec_BW        1.3280 # total instructions (mis-spec + committed) per cycle
sim_IPB            4.7033 # instruction per branch
```

Spec Benchmarks

- Set of scientific applications from that Standard Performance
Evaluation Corporation
- Compiled in SimpleScalar Binary Form
- Spec95
<http://www.cs.wisc.edu/mscalar/ss/>
- Spec2000
<http://www.simplescalar.com/benchmarks.html>



Global Simulator Options

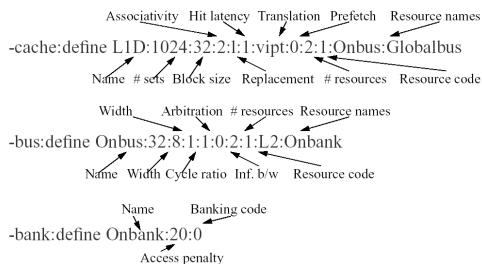
- {all simulators}
 - ❑ -h -print simulator help message
 - ❑ -d -enable debug message
 - ❑ -i -start up in DLite! Debugger
 - ❑ -q -quit immediately (use w/ dumpconfig)
 - ❑ -config <file> -read config parameters from <file>
 - ❑ -dumpconfig<file> -save config parameters from <file>
- Run Program without parameters to find other options available



Configuration Files

- Put complex command line options into a file
- To generate a configuration file:
 - Specify non-default options on command line
 - Include “- dumpconfig <file>” to generate configuration file
- Reload configuration files using “-config <file>” command line option
- “#” is interpreted as a comment

Defining A Memory Hierarchy



Specifying the Branch Predictor

- Specifying the branch predictor type:
-bpred <type>
- The supported predictor types are:

nottaken	always predict not taken
taken	always predict taken
perfect	perfect predictor
bimod	bimodal predictor (BTB w/ 2 bit counters)
2lev	2-level adaptive predictor

DLite!, the Light Debugger

- Lightweight symbolic debugger
- Not supported on sim-fast
- Start simulator with "-i" option
- DLite! expressions may include:
 - Operators: +, -, /, *
 - Literals: 10, 0xff, 077
 - Symbols: main, vprintf
 - Registers: e.g. \$r1, \$f4, \$pc, \$lo



DLite! Main Features

- break, dbreak, rbreak
 - Set text, data, and range breakpoints
- regs, iregs, fregs
 - Display all integer and FP register state
- dump <addr> <count>
 - Dump <count> bytes of memory at <addr>
- dis <addr> <count>
 - Disassemble <count> insts starting at <addr>
- print <expr>, display <expr>
 - Display expression or memory
- mstate: display machine-specific state
 - Mstate alone displays options, if any

DLite! Breakpoints

- Breakpoints:
 - Code:
 - `break <addr>`, e.g. `break main`, `break 0x400148`
 - Data:
 - `dbreak <addr> {r|w|x}`
 - `r=read, w=write, x=execute`, e.g., `dbreak stdin w`, `dbreak sys_count wr`
 - Range:
 - `rbreak <range>`, e.g., `rbreak @main:+279`, `rbreak 2000:3500`

SimpleScalar Version 3.0

- Memory Extensions
- MultiProcessor simulator
- Value prediction/ trace caches
- Max-inst option
- More instruction sets
- For more info: Download and view [readme](#) from SimpleScalar WebSites
